```
    public String getOwner() { return owner; }
    public int getPoints() { return points; }
    public int getStatusPoints() { return statusPoints; }
}
```

**The table is read-only, so you only need getters, not setters, to access the field values.**

Cucumber lets you pass a list of `Accounts` directly to the step definition, as shown here:

```
@Given("^the following accounts:$")
public void the_following_accounts(List<Account> accounts) {
    InTestDatabase.createAccounts(accounts);
}
```

This will automatically use the table header values to create a list of matching `Account` objects.

### 6.5.5 Step definitions for tables of examples

Step definitions for example tables are easy to implement in Cucumber-JVM. In fact, because the step definitions are based on regular expressions, there's no difference between a step definition written for a normal scenario step and one written for an example-based step.

Suppose you're automating the following scenario in Cucumber-JVM:

```
Scenario Outline: Earning extra points on flights by Frequent Flyer status
  Given I am a <status> Frequent Flyer member
  When I fly on a flight that is worth <base> base points
  Then I should earn a status bonus of <bonus>
  And I should have guaranteed minimum earned points per trip of <minimum>
  And I should earn <total> points in all
Examples:
  | status   | base | bonus | minimum | total | notes
  | Standard | 439  | 0     | 0       | 439   |
  | Silver   | 439  | 220   | 500     | 659   | 50% bonus
  | Silver   | 148  | 111   | 500     | 500   | minimum points
  | Gold     | 474  | 400   | 1000    | 1000  | minimum points
  | Gold     | 2041 | 1531  | 1000    | 3572  | 75% bonus
```

You could use the following step implementation to match the first line of this scenario:

```
@Given("I am a (.*) Frequent Flyer member")
public void useAMemberWithAGivenStatus(String status) {
    member = members.getMember().withStatus(status);
}
```

**The regular expression will match steps in a table or in a conventional scenario.**

This makes it easy to reuse step definitions between table-based and conventional scenarios.

### 6.5.6 Sharing data between steps

Cucumber provides a simple and convenient way to share data between steps using dependency injection. You can create a helper class to store information you want to share between steps, and then inject it into the constructor of each step definition class that uses this information.